Introduction to Python

Session 11

Some Useful Modules
copy, re, argparse, random

Modules are used via import

Copy an object: `copy`

`==` vs. `is`

both list are allocated in a different position in memory

```
list1 = [ set(), set(), set(), set() ]

list2 = [ set(), set(), set(), set() ]

list1 == list2
True

list1 is list2
False
```

Compares the two object instances using the **__eq__ method**

Compares if the two objects are the same object instance. Equivalent to: **`id(list1)==id(list2)`**

because we have created

Copy an object: `copy`

- **`==`** is different to **`is`**
- **`is`** and **`is not`** is usually used to check if a name is assigned to the **`None`** constant
- **`None`** is the sole value of the type **`NoneType`**
- All names having the **`None`** value refer to the same instance
- The same can be applied to **`True`** and **`False`** values of the **`bool`** type

because we can only have one value for True and False

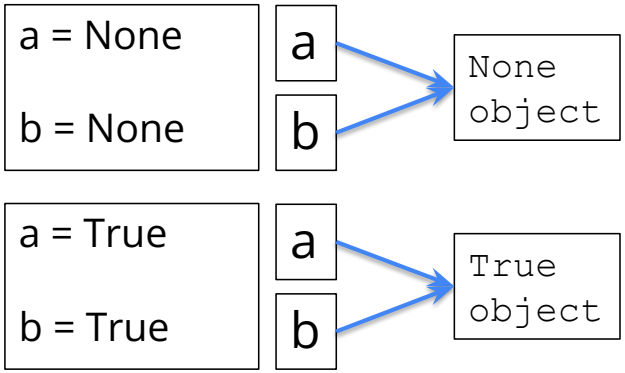
```
if my_instance_name is None:
    …

if my_instance_name is not None:
    …
```

None is always the same value
we cannot have two different None values

Copy an object: `copy`

- **`==`** is different to **`is`**

- **`is`** statement is usually applied to:
  - **`my_variable is None`**
  - **`my_variable is False`**
  - **`my_variable is True`**

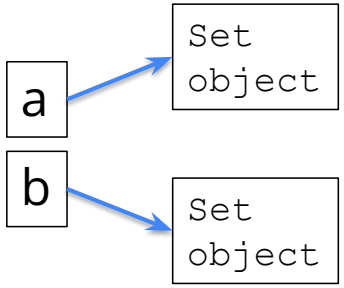



```
if my_instance_name is True:
    …

if my_instance_name is not False:
    …
```

Copy an object: `copy`

- **Variable assignment:** with this syntax, the object resulting from the right expression is assigned to the given name

```
a = set([1, 2, 3])

b = set([1, 2, 3])

b is a
False
```



a = 2.2
b= 2.2.
a is b
false

we have created two instances of floats
For int when we do the same

a = 1
b = 1

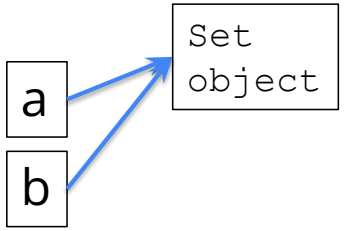
a is b
True

python uses them as singletons, so for int always use == instead

Copy an object: `copy`

- **Variable assignment:** with this syntax, the object resulting from the right expression is assigned to the given name

```
a = set([1, 2, 3])

b = a

b is a
True
```

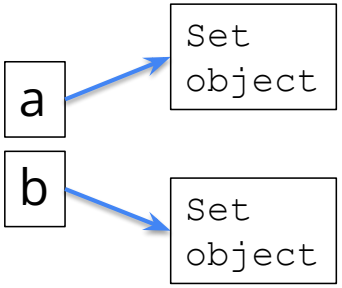


How can we create a **copy** of an object?

A new instance of the same class with the same attribute values

Copy an object: `copy`


```
a = set([1, 2, 3])

b = copy.copy(a)

b is a
False

b == a
True
```



a = set([1,2,3,])
b = set([1,2,3,])

a == b
True

a is b
False

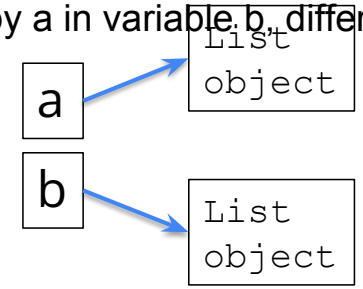
if we want to create another copy to assign it into another variable, and for that we have the copy.copy(b) function. This is useful when you want to copy your object.

Copy an object: `copy`

- **Shallow vs. deep copy:**

let's create lists again with three empty sets. Copy a in variable b, different ids so that's why b is a returns False

```
a = [ set(), set(), set() ]

b = copy.copy(a)

b is a
False

b[0] is a[0]
True
```



The list object is copied but the list object contains references to other objects (3 sets in this example).

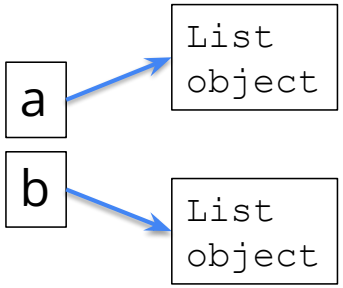
Object instances referenced inside the “copied” instance are not duplicated.

because is a shallow copy. If we want to copy what is inside the list and not only the list itself, we have copy.deepcopy function.

Copy an object: `copy`

- **Shallow vs. deep copy:**

```
a = [ set(), set(), set() ]

b = copy.deepcopy(a)

b is a
False

b[0] is a[0]
False
```



The list object is copied.
All referenced objects are copied too

Copy an object: `copy`


```
class T(object):
    my_class_attribute = set()

    def __init__(self):
        self.instance_list = []

t1 = T()
t2 = t1

t2 is t1
True

t2 = copy.copy(t1)
t2 is t1
False

t1.instance_list is t2.instance_list
True

t1.my_class_attribute is t2.my_class_attribute
True
```

we have created a new class with an attribute of that class called instance_list.

from S11 import T:
type(T)
class `'type' --> so it's type class

creation of instances of that class

t2 is pointiog to the same object which is t1

t1 and t2 are different objects, but their attributes are the same object instances: **shallow copy**

creating a shallow copy

the different attriubte poitns ot the same object, because is a shallow copy.

Copy an object: copy


```
class T(object):
    my_class_attribute = set()

    def __init__(self):
        self.instance_list = []

t1 = T()
t2 = t1

t2 is t1
True

t2 = copy.deecopy(t1)
t2 is t1
False

t1.instance_list is t2.instance_list
False

t1.my_class_attribute is t2.my_class_attribute
True
```

t1 and t2 are different objects, their data attributes are different objects: **Deep copy**. They only share class attributes.

now the instance have been also ben copied, so this yields different objects.

this is still true, because all instances of the class share the same attributes

Regular Expressions

Regular expressions: `re`

Regular expression: sequence of characters that forms a search pattern.

import re → in python script

re.compile("ab*") #this means anything starting from ab and whatever

Pattern matching:

```
. ^ $ * + ? { } [ ] \ | ( )
\d \D \w \W \s \S
...
```

To have a look in the syntax of regular expressions:
http://docs.python.org/3/howto/regex.html#regex-howto

Regular expressions: `re`

Regular expression: sequence of characters that forms a search pattern.

```
import re
```

To have a look in the syntax of regular expressions:
http://docs.python.org/3/howto/regex.html#regex-howto

Regular expressions: `re`

Pattern object (SRE_Pattern object)

```
import re

p = re.compile("ab*")

print(p)
<_sre.SRE_Pattern object at 0x109899608>
```

To have a look in the syntax of regular expressions:
http://docs.python.org/3/howto/regex.html#regex-howto

Regular expressions: `re`

Pattern object (SRE_Pattern object)

```
import re

p = re.compile("ab*", re.IGNORECASE)

print(p)
<_sre.SRE_Pattern object at 0x109899608>
```

we have different flags, like IGNORCASE, match without taking into account the case

flags:

```
IGNORECASE,
MULTILINE,
DOTALL
VERBOSE
…
```

To have a look in the syntax of regular expressions:
http://docs.python.org/3/howto/regex.html#regex-howto

Regular expressions: `re`

`Pattern object` methods:

- **`search()`:** Scan through a string, looking for any **location** where this matches. Returns a **`MatchObject`** instance if it is found. If not, return **`None`**.
- **`match()`:** Determine if the RE matches at the beginning of the string.
- **`findall()`**: Find all substrings where the RE matches and return them as a **list of strings.**
- **`finditer()`**: Find all substrings where the RE matches, and return them as an **Iterator of `MatchObjects`**
- **`split()`:** splits a string using where the RE matches

Regular expressions: `re`

`MatchObject` methods:

- `group()`: Return the string matched by the RE
- `start()`: Return the index of the starting position of the match.
- `end()`: Return the index of the ending position of the match.
- `span()`: Return a tuple (start, end) of the match.

upf.

Regular expressions: `re`


```
>>> import re

>>> my_string = "the text: 12 drummers drumming, 11 pipers piping, 10
lords a-leaping"

>>> p = re.compile("\d+")
```

this stands for any digit, repeated one or more times

```
>>> m = p.match(my_string)

>>> if m is None:
...     print("Match not found")
... else:
...     print(m)
...
Match not found
```

match only looks at the beginning of the string !

`\d`

For Unicode (str) patterns:

Matches any Unicode decimal digit (that is, any character in Unicode character category [Nd]). This includes `[0-9]`, and also many other digit characters. If the `ASCII` flag is used only `[0-9]` is matched.

For 8-bit (bytes) patterns:

Matches any decimal digit; this is equivalent to `[0-9]`.

Regular expressions: `re`


```
>>> import re

>>> my_string = "the text: 12 drummers drumming, 11 pipers piping, 10
lords a-leaping"

>>> p = re.compile("\d+")

>>> m = p.match(my_string)

>>> if m is None:
...     print("Match not found")
... else:
...     print(m)
...
Match not found
```

match only looks at the beginning of the string !

+

Causes the resulting RE to match 1 or more repetitions of the preceding RE. `ab+` will match 'a' followed by any non-zero number of 'b's; it will not match just 'a'.

Regular expressions: `re`


```
>>> import re

>>> my_string = "the text: 12 drummers drumming, 11 pipers piping, 10
lords a-leaping"

>>> p = re.compile("\d+")
```

(r"\d+") --> r tells python not to interpret this as a string

```
>>> m = p.search(my_string)
```

search looks for the first occurrence along all the string

```
>>> if m is None:
...     print("Match not found")
... else:
...     print m
...
<_sre.SRE_Match object at 0x1098a01d0>

>>> print m.group()
12
```

group returns the first match

Regular expressions: `re`

Example: **`findall`**

```
>>> import re

>>> my_string = "the text: 12 drummers drumming, 11 pipers piping, 10
lords a-leaping"

>>> p = re.compile("\d+")

>>> m = p.findall(my_string)

>>> p.findall(my_string)
['12', '11', '10']
```

Returns a list with all the occurrences.

Regular expressions: `re`

Example: `finditer`

```
>>> import re

>>> my_string = "the text: 12 drummers drumming, 11 pipers piping, 10
lords a-leaping"

>>> p = re.compile("\d+")

>>> iterator = p.finditer(my_string)

>>> print(iterator)
<callable-iterator object at 0x1098b6b90>

>>> for match in iterator:
...     print("%s: %s" %(match.group(), match.span()))
...
12: (10, 12)
11: (32, 34)
10: (50, 52)
```

Returns an iterator of MatchObjects

Regular expressions: `re`

Example: **Using regular expressions without creating pattern objects**

```
>>> import re

>>> my_string = "the text: 12 drummers drumming, 11 pipers piping, 10
lords a-leaping”

>>> re.search("\d+", my_string)
<_sre.SRE_Match object at 0x1098a0168>

>>> re.match("\d+",my_string)

>>> re.findall("\d+",my_string)
['12', '11', '10']
```

If the regular expression is going to be used repetitive times (for example, checking for the pattern in each line of a file), it is better to create the Pattern Object with the compile function: Python compiles only once the regular expression.

compile the reg ex in a compiled object and use it

Regular expressions: `re`

Grouping:

```
>>> import re

>>> my_string = "abcdefghijk”

>>> p = re.compile("abc(def)gh(ij)k")

>>> m = p.match(my_string)

>>> if m:
...     print(m.groups())
...
('def', 'ij')
```

define groups in parenthesis

Regular expressions: `re`

Grouping:

```
>>> import re

>>> my_string = "abcdefghijk”

>>> p = re.compile("abc(def)gh(ij)k")

>>> m = p.match(my_string)

>>> print(m.group(0))
abcdefghijk
>>> print(m.group(1))
def
>>> print(m.group(2))
ij
```

All RE matched

First group matched

Second group matched

Regular expressions: `re`

Grouping:

```
>>> import re

>>> my_string = "abcdefghijk”

>>> p = re.compile(r"abc(def)gh(ij)k")

>>> m = p.match(my_string)

>>> m.span(0)
(0, 11)
>>> m.span(1)
(3, 6)
>>> m.span(2)
(8, 10)
```

Start and end from all RE pattern matched

Start and end from first group

Start and end from second group

Regular expressions: `re`

Backreferences: Specify contents of an earlier capturing group.

\b empty string
\w + any character from a word one or more times
\s+ space one or more times
\1 group nº one

Example: Detect repeated consecutive words.

Group number

```
>>> p = re.compile(r"(\b\w+)\s+\1")

>>> p.search('Paris in the the spring').group()
'the the'
```

`\b`

Matches the empty string, but only at the beginning or end of a word. A word is defined as a sequence of word characters. Note that formally, `\b` is defined as the boundary between a `\w` and a `\W` character (or vice versa), or between `\w` and the beginning/end of the string. This means that `r'\bfoo\b'` matches `'foo'`, `'foo.'`, `'(foo)'`, `'bar foo baz'` but not `'foobar'` or `'foo3'`.

Regular expressions: `re`

Backreferences: Specify contents of an earlier capturing group.

Example: Detect repeated consecutive words.

Group number

```
>>> p = re.compile(r"(\b\w+)\s+\1")

>>> p.search('Paris in the the spring').group()
'the the'
```

`\s`

For Unicode (str) patterns:

Matches Unicode whitespace characters (as defined by `str.isspace()`). This includes `[ \t\n\r\f\v]`, and also many other characters, for example the non-breaking spaces mandated by typography rules in many languages.

Matches `[ \t\n\r\f\v]` if the `ASCII` flag is used.

Replacing: sub

```
>>> p = re.compile( '(blue|white|red)' )
```

replace each of the matches of the pattern with yellow

```
>>> p.sub( 'yellow', 'blue socks and red shoes')
'yellow socks and yellow shoes'

>>> p.sub( 'yellow', 'blue socks and red shoes', count=1)
'yellow socks and red shoes'
```

Replacement string.
It can be a function name.

Maximum number of pattern occurrences to be replaced

Replacing: subn

```
>>> p = re.compile( '(blue|white|red)' )
```

returns the number of replacements it did

```
>>> p.subn( 'yellow', 'blue socks and red shoes')
('yellow socks and yellow shoes', 2)

>>> p.subn( 'yellow', 'blue socks and red shoes', count=1)
('yellow socks and red shoes', 1)
```

Returns a tuple with the modified string and the number of replacements

Parsing command line arguments.

`argparse`

Command line arguments are stored in the list **`sys.argv`**

Some modules make the processing of input arguments easier: **argparse**

Manage arguments in our scripts.

http://docs.python.org/3/library/argparse.html

argparse

import argparse

```
parser = argparse.ArgumentParser(description="This program does BLA
                                                        BLA BLA")

parser.add_argument('-i', '--input',
                    dest = "infile",
                    action = "store",
                    default = None,
                    help = "Input file")

parser.add_argument('-o', '--output',
                    dest = "outfile",
                    action = "store",
                    default = "default_output.txt",
                    help = "outputfile")

options = parser.parse_args()
```

Argument short and long names

Help to be printed

Destination name

Default value

argparse

Create an ArgumentParser object instance

Description of the program

```
parser = argparse.ArgumentParser(description="This program does BLA
                                                        BLA BLA")

parser.add_argument('-i', '--input',
                    dest = "infile",
                    action = "store",
                    default = None,
                    help = "Input file")

parser.add_argument('-o', '--output',
                    dest = "outfile",
                    action = "store",
                    default = "default_output.txt",
                    help = "outputfile")

options = parser.parse_args()
```

in which variable we are storing the argument

store that

no defualt value

knows what to put there

Define arguments

the same for output

Parse arguments and set them in a new namespace

at the end we use the pars_args() method to create objects

argparse


```
SYSTEM> python my_stript.py -h
```

```
usage: test_argparse.py [-h] [-i INFILE] [-o OUTFILE]

This program does BLA BLA BLA

optional arguments:
  -h, --help            show this help message and exit
  -i INFILE, --input INFILE
                        Input file
  -o OUTFILE, --output OUTFILE
                        outputfile
```

Help printed when the program is executed with incorrect arguments or with the `-h` or `-help` option

argparse


```
parser = argparse.ArgumentParser(description="This program does BLA
                                                        BLA BLA")

parser.add_argument('-i', '--input',
                    dest = "infile",
                    action = "store",
                    default = None,
                    help = "Input FASTA formatted file")

parser.add_argument('-o', '--output',
                    dest = "outfile",
                    action = "store",
                    default = "default_output.txt",
                    help = "outputfile")

options = parser.parse_args()

options.infile
options.outfile
```

Accessing the arguments' values

python S11.py -h

argparse


```
parser = argparse.ArgumentParser(description="This program does BLA
                                                        BLA BLA")

parser.add_argument('-i', '--input',
                    dest = "infile",
                    action = "store",
                    default = None,
                    help = "Input FASTA formatted file")

parser.add_argument('-v', '--verbose',
                    dest = "verbose",
                    action = "store_true",
                    default = False,
                    help = "Print log in stderr")

options = parser.parse_args()

options.infile
options.verbose
```

Save True or False if the argument is used

argparse


```
parser = argparse.ArgumentParser(description="This program does BLA
                                                        BLA BLA")

parser.add_argument('-i', '--input',
                    dest = "infile",
                    action = "store",
                    default = None,
                    help = "Input FASTA formatted file")

parser.add_argument('-l', '--loops',
                    dest = "number_of_loops",
                    action = "store",
                    default = 0,
                    type = int,
                    help = "Integer specifying the number of loops")
options = parser.parse_args()

options.infile
Options.number_of_loops
```

we can access the name we have put here

Force the argument to be an integer. Casting to the type is done automatically. If it cannot be done, raise an exception

ArgumentParser class

```
parser = argparse.ArgumentParser(description="This program
                                        does BLA BLA BLA")
```

Other parameters:

- **prog**: the name of the program
- **usage**: the string describing the program usage
- **description**: Text to display before the argument help
- etc.

Adding arguments

```
parser.add_argument( … )
```

Available parameters:

- **name or flags**
- **action**
- **default**: default value if the argument is not used.
- **type**: type conversion to be done.
- **required**: whether or not the command-line option may be omitted.
- **choices**: container with the allowable values of the argument.
- **dest**: name of the attribute to be added in the namespace created by parse_args()
- etc.

Pseudo random numbers

```
import random
```

https://docs.python.org/3/library/random.html

Random: floats.
Uniform distributions:

how to create random numbers

- `random()`: return the next random floating point number in the range [0.0, 1.0)

- `randint(a,b)`: returns a random integer between a and b, both included.

random.randit(0,10)

- **uniform(a,b):** return a random floating point number between a and b, both included.

random.uniform(0,10)

- etc.

Random: floats.
Specific distributions:

- `gauss(mu, sigma)` from gaussian distribution
- `gammavariate(alpha, beta)`
- etc.

Random: lists

- **`choice`**: return a random element from a non-empty sequence.

- **`sample(population, k)`**: return a k length list of elements chosen from the population sequence. Used for random sampling without replacing.

- **`shuffle`**: shuffle elements in a list IN PLACE

random

Create a list of 20 random integers between [0,100]

```
>>> my_random_list = [ random.randint(0,100) for x in range(20) ]
```

this will create 20 random int from 0-20

```
>>> my_random_list
[96, 79, 38, 58, 36, 55, 18, 95, 87, 20, 7, 28, 26, 56, 48, 35, 26,
28, 6, 39]
```

the method sample is to get random samples from that list.

```
>>> random.sample(my_random_list,5)
[28, 7, 55, 39, 36]

>>> random.sample(my_random_list,5)
[79, 28, 26, 48, 95]

>>> random.shuffle(my_random_list)
>>> my_random_list
[79, 26, 56, 48, 96, 39, 55, 87, 6, 28, 38, 7, 36, 35, 58, 26, 20, 95,
18, 28]
```

in this case random subsets of 5 elements

Create two different random samples of 5 elements from the initial list

Shuffle the elements from the initial list

gzip

```
import gzip
```

http://docs.python.org/3/library/gzip

Read/write gzipped files

Open a gzipped file in reading mode. Returns a file object.

```
import gzip
import sys

with gzip.open("my_gzipped_file.txt.gz", "r") as fd:

    for line in fd:
        sys.stdout.write(line)


out_fd = gzip.open("my_output_file.txt.gz", "wt")
out_fd.write("STRING TO WRITE COMPRESSED\n")
out_fd.close()
```